Verifying sample relationships using DNA Methylation data en Whole Genome Sequencing data

true

2017-07-13

Abstract

Here we will show how you could verify sample relationships on publicly available DNA methylation data and whole genome sequencing data. We use the beta-value matrix extracted from GEO (GSE39672) and collect 1000G genotypes for partially overlapping samples from IGSR: The International Genome Sample Resource.

Contents

Methylation Data **1**

Genotype Data **4**

Genotype and Methylation Data **9**

Detect mislabelled samples **13**

SessionInfo **15**

Reference **17**

Methylation Data

First we extract the data from GEO using the *GEOquery*-package and beta-values are extracted using the `exprs`-function:

```
library(GEOquery)
gset <- getGEO("GSE39672", GSEMatrix=TRUE)
```

```
## https://ftp.ncbi.nlm.nih.gov/geo/series/GSE39nnn/GSE39672/matrix/
```

```
## OK
```

```
## Found 1 file(s)
```

```
## GSE39672_series_matrix.txt.gz
```

```
## Using locally cached version: /tmp/RtmpIFBy5U/GSE39672_series_matrix.txt.gz
```

```
## Using locally cached version of GPL13534 found here:
## /tmp/RtmpIFBy5U/GPL13534.soft

## Warning in read.table(file = file, header = header, sep = sep, quote =
## quote, : not all columns named in 'colClasses' exist
```

```
gset
```

```
## $GSE39672_series_matrix.txt.gz
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 485577 features, 133 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: GSM977267 GSM977268 ... GSM977399 (133 total)
##   varLabels: title geo_accession ... data_row_count (37 total)
##   varMetadata: labelDescription
## featureData
##   featureNames: cg00000029 cg00000108 ... rs9839873 (485577 total)
##   fvarLabels: ID Name ... SPOT_ID (37 total)
##   fvarMetadata: Column Description labelDescription
## experimentData: use 'experimentData(object)'
## Annotation: GPL13534
```

```
betas <- exprs(gset[[1]])
dim(betas)
```

```
## [1] 485577    133
```

Sample names can be extract from the gse-object using the pData-function:

```
population <- gsub("^.* ", "", pData(gset[[1]])$characteristics_ch1)
table(population)
```

```
## population
## CEU YRI
##  60  73
```

```
colnames(betas) <- gsub("^.* ", "", pData(gset[[1]])$title)
betas[1:5, 1:5]
```

```
##            NA19147 NA12750 NA18916 NA12146 NA12057
## cg00000029 0.64883 0.49165 0.59442 0.70837 0.48281
## cg00000108 0.85525 0.88946 0.90310 0.89617 0.91059
## cg00000109 0.78733 0.87134 0.67031 0.80378 0.64081
## cg00000165 0.27610 0.28159 0.57978 0.32332 0.52309
## cg00000236 0.67505 0.81193 0.77600 0.76603 0.81761
```

If we would have had the raw idats we could extract the 65 SNPs. However, experiences show that the 65 SNPs available on the array are not enough to perform sample verification with high confidence. Fortunately,

several probes on the array contain SNPs occuring frequently in different populations(Chen et al. 2013; Zhou, Laird, and Shen 2016).

We have made the data available from the package *DNAmArray*.

Now we make a selection of CpGs probably affected by polymorphic SNPS in these two populations:

```
library(DNAmArray)
data(hm450.manifest.pop.GoNL)
cpgs <- names(hm450.manifest.pop.GoNL[mcols(hm450.manifest.pop.GoNL)$MASK.snp5.CEU
                                      | mcols(hm450.manifest.pop.GoNL)$MASK.snp5.YRI])
```

Next the beta-values are converted to genotypes using our enhanced K-means algorithm:

```
library(omicsPrint)
dnamCalls <- beta2genotype(betas[rownames(betas) %in% cpgs, ])
dim(dnamCalls)
```

```
## [1] 1060  133
```

```
dnamCalls[1:5, 1:5]
```

```
##            NA19147 NA12750 NA18916 NA12146 NA12057
## cg00009523       3       2       3       2       2
## cg00018828       3       2       3       2       1
## cg00021325       1       3       2       1       1
## cg00025680       1       2       2       3       1
## cg00033213       3       3       2       1       1
```

The DNA methylation based genotype calls can directly supplied to the allelesharing algorithm to perform the intra-omic sample matching:

```
data <- alleleSharing(dnamCalls)
```

```
## Hash relations

## There are 0 SNP dropped because of low call rate!

## There are 0 sample set to NA because too little SNPs called!

## Using 1060 polymorphic SNPs to determine allele sharing.

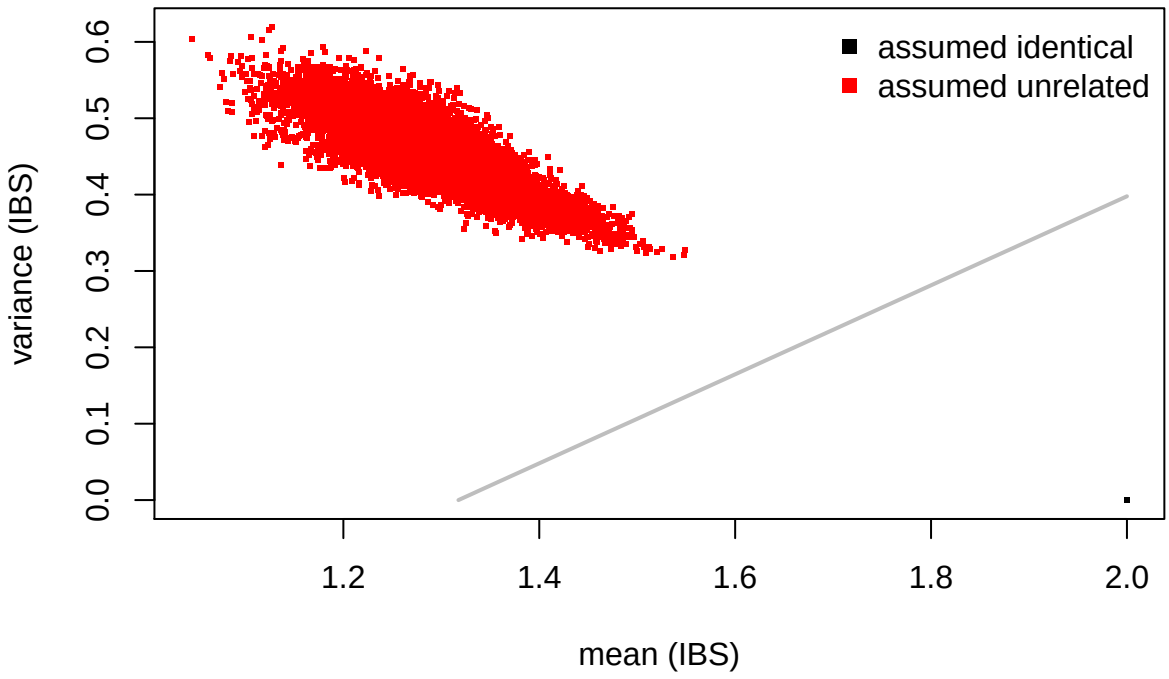
## Running `square` IBS algorithm!

## 134 of 8911 (1.5%) ...

## 8351 of 8911 (93.72%) ...
```

```
mismatches <- inferRelations(data)
```

```
##                    Assumed relation
## Predicted relation identical unrelated
##          identical       133         .
##          unrelated         .      8778
```



There are no unknown relations in the data, verifying Moen et al. (2013), as all samples should be unrelated.

Genotype Data

Since the sample used in this study (GSE39672) are overlapping with the HapMap and 1000G samples we can extract the whole genome sequencing derived genotypes.

Here is a helper-function for the extraction of genotypes from the 1000G per chromosome vcf-files using the *VariantAnnotation*-package. Furthermore, we use the `genotypeToSnpMatrix`-function to convert genotypes that we directly can use in the allelesharing algorithm.

```
getSnpMatrix <- function(chr, samples = NULL, which) {
    require(VariantAnnotation)

    message("Extracting SNPs for chromosome: ", chr)

    vcfFile <- paste0("ftp://ftp.1000genomes.ebi.ac.uk/vol1/ftp/release/20130502/ALL.chr", chr,
```

```
                    ".phase3_shapeit2_mvncall_integrated_v5a.20130502.genotypes.vcf.gz")

    indexFile <-  paste0("ftp://ftp.1000genomes.ebi.ac.uk/vol1/ftp/release/20130502/ALL.chr", chr,
                         ".phase3_shapeit2_mvncall_integrated_v5a.20130502.genotypes.vcf.gz.tbi")

    tabFile <- TabixFile(vcfFile, indexFile)

    if(is.null(samples))
        param <- ScanVcfParam(geno = "GT",
                              which = which[seqnames(which) == chr])
    else
        param <- ScanVcfParam(geno = "GT",
                              samples = samples,
                              which = which[seqnames(which) == chr])

    message("Reading vcf ...")
    vcf <- readVcf(tabFile, "hg19", param = param)

    message("Transform to SnpMatrix ...")
    m <- genotypeToSnpMatrix(vcf)$genotypes

    ##debugging
    ##message(str(m))

    m <- t(matrix(as.numeric(m),
                  nrow = nrow(m),
                  ncol = ncol(m),
                  dimnames = dimnames(m)))

    rownames(m) <- paste(seqnames(vcf@rowRanges),
                         start(vcf@rowRanges), sep=":")
    m
}
```

Extract genotypes from the vcf-files that are occuring in all populations. Optionally you could extract only those that overlap with the DNA methylation data by suppling the `sample`-names. Now we use a `GenomicRange`-object to specify specifically which SNPs we would like to extract.

This extraction can take some time. We run this on a multi-core machine using 22 cores in combination with the *BiocParallel*-package.

```
library(DNAmArray)
data(hm450.manifest.pop.GoNL)
gr <- hm450.manifest.pop.GoNL[mcols(hm450.manifest.pop.GoNL)$MASK.snp5.CEU
                                    | mcols(hm450.manifest.pop.GoNL)$MASK.snp5.YRI]

mcols(gr) <- NULL
seqlevels(gr) <- mapSeqlevels(seqlevels(gr), "NCBI")
gr
```

```
## GRanges object with 36755 ranges and 0 metadata columns:
##              seqnames                ranges strand
##                 <Rle>             <IRanges>  <Rle>
```

```
##   cg08477687        1       [566570, 566571]      *
##   cg00645010        1       [567113, 567114]      *
##   cg06402284        1       [762592, 762593]      *
##   cg20788133        1       [765028, 765029]      *
##   cg09139287        1       [787398, 787399]      *
##          ...      ...                     ...    ...
##   cg12531767        X [153644319, 153644320]      *
##   cg13766601        X [153657086, 153657087]      *
##   cg10645377        X [153659893, 153659894]      *
##   cg04811950        X [153666533, 153666534]      *
##   cg08873063        X [153776107, 153776108]      *
##   -------
##   seqinfo: 25 sequences from an unspecified genome; no seqlengths
```

```
##check one
##dnaCalls <- getSnpMatrix(22, samples = samples, which=gr)
##dim(dnaCalls)
library(BiocParallel)
register(MulticoreParam(22))
##dnaCalls <- bplapply(1:22, getSnpMatrix, samples = colnames(dnamCalls), which=gr)
dnaCalls <- bplapply(1:22, getSnpMatrix, samples = NULL, which=gr)
dnaCalls <- do.call('rbind', dnaCalls)
dim(dnaCalls)
```

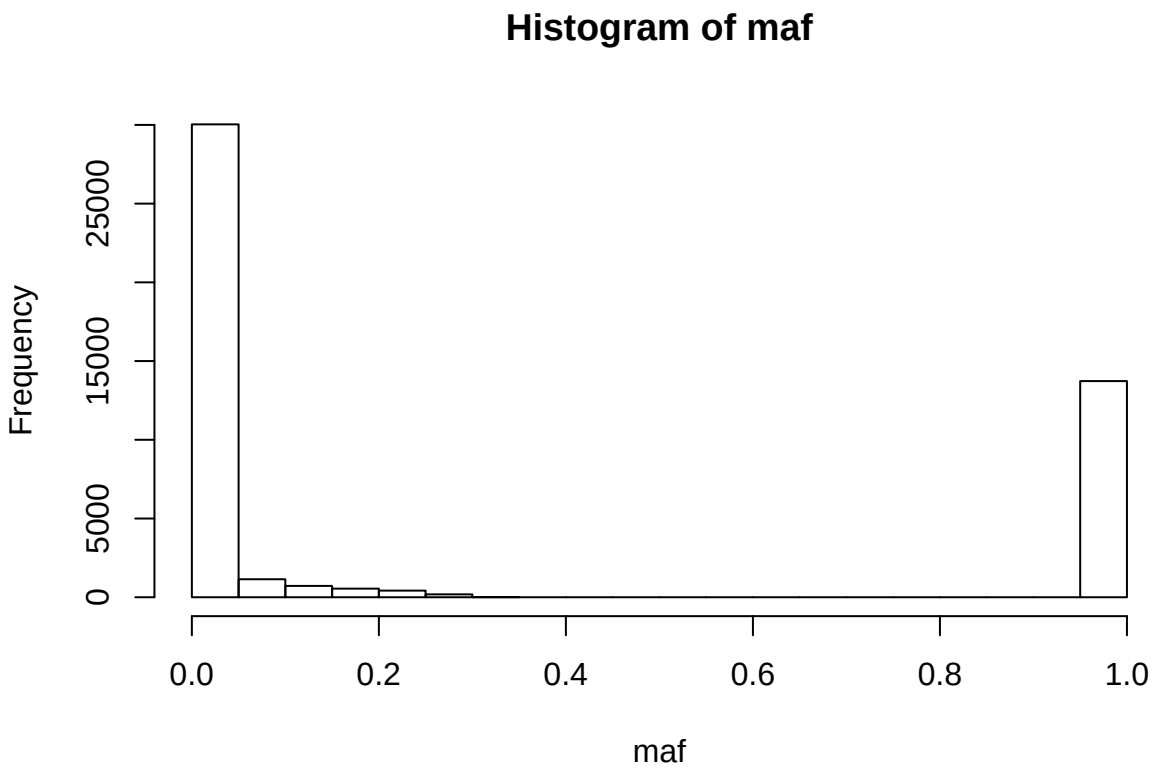
```
## [1] 46786  2504
```

```
dnaCalls[1:5, 1:5]
```

```
##          HG00096 HG00097 HG00099 HG00100 HG00101
## 1:668630       0       0       0       0       0
## 1:762592       2       3       2       3       3
## 1:668630       0       0       0       0       0
## 1:765028       1       1       1       1       1
## 1:668630       0       0       0       0       0
```

To select the most informative SNPs, e.g., we can calculate the minor allel frequenties and retain those above a certain threshold, i.e., 0.01 and drop uninformative ones.

```
maf <- apply(dnaCalls, 1, function(x) min(table(x)/length(x)))
hist(maf)
```




```
dnaCalls <- dnaCalls[maf > 0.01 & maf < 1,]
dim(dnaCalls)
```

```
## [1] 9647 2504
```

Now we can run the allelesharing algorithm on the genotype data:

```
library(omicsPrint)
data <- alleleSharing(dnaCalls)
```

```
## Hash relations

## There are 0 SNP dropped because of low call rate!

## There are 0 sample set to NA because too little SNPs called!

## Using 9647 polymorphic SNPs to determine allele sharing.

## Running `square` IBS algorithm!

## 2505 of 3136260 (0.08%) ...

## 245451 of 3136260 (7.83%) ...
```

```
## 480901 of 3136260 (15.33%) ...

## 706351 of 3136260 (22.52%) ...

## 921801 of 3136260 (29.39%) ...

## 1127251 of 3136260 (35.94%) ...

## 1322701 of 3136260 (42.17%) ...

## 1508151 of 3136260 (48.09%) ...

## 1683601 of 3136260 (53.68%) ...

## 1849051 of 3136260 (58.96%) ...

## 2004501 of 3136260 (63.91%) ...

## 2149951 of 3136260 (68.55%) ...

## 2285401 of 3136260 (72.87%) ...

## 2410851 of 3136260 (76.87%) ...

## 2526301 of 3136260 (80.55%) ...

## 2631751 of 3136260 (83.91%) ...

## 2727201 of 3136260 (86.96%) ...

## 2812651 of 3136260 (89.68%) ...

## 2888101 of 3136260 (92.09%) ...

## 2953551 of 3136260 (94.17%) ...

## 3009001 of 3136260 (95.94%) ...

## 3054451 of 3136260 (97.39%) ...

## 3089901 of 3136260 (98.52%) ...

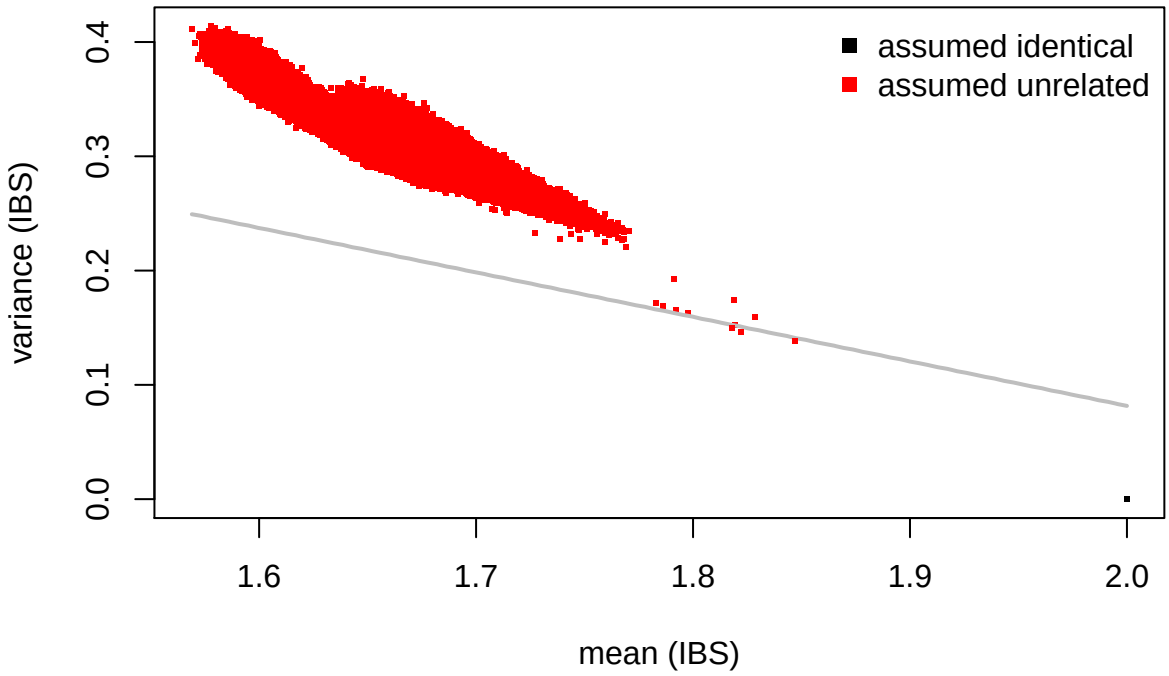
## 3115351 of 3136260 (99.33%) ...

## 3130801 of 3136260 (99.83%) ...

## 3136251 of 3136260 (100%) ...
```

```
mismatches <- inferRelations(data)
```

```
##                   Assumed relation
## Predicted relation identical unrelated
##          identical      2504         3
##          unrelated         .   3133753
```




```
mismatches
```

```
##             mean       var colnames.x colnames.y  relation predicted
## 1907749 1.846792 0.1386645    HG02479    HG02429 unrelated identical
## 2554683 1.818182 0.1496051    HG03754    HG03750 unrelated identical
## 3133426 1.822017 0.1465274    NA20900    NA20882 unrelated identical
```

A few possibly family relations seems to appear around mean IBS of 1.8 and variance 0.15.

Genotype and Methylation Data

To be able to run the allele sharing algorithm between the DNA methylation and genotype data there should be an overlapping set of SNPs.

Here we map probe identifiers to `chromosome:position`, like the genotype data using information for the array provider available from the *FDb.InfiniumMethylation.hg19*-package, and rename to probes.

```
library(FDb.InfiniumMethylation.hg19)
rngs <- getPlatform(platform='HM450', genome='hg19')
```

```
## Fetching coordinates for hg19...
```

```
seqlevels(rngs) <- mapSeqlevels(seqlevels(rngs), "NCBI")

map <- names(rngs)
names(map) <- paste(seqnames(rngs),
                    mcols(rngs)$probeTarget, sep=":")
mid <- match(map, rownames(betas))
rownames(betas)[mid] <- names(map)

library(omicsPrint)
dnamCalls <- beta2genotype(betas[rownames(betas) %in% rownames(dnaCalls),])
dim(dnamCalls)
```

```
## [1] 227 133
```

```
data <- alleleSharing(dnamCalls, dnaCalls, phasing=TRUE)
```

```
## Hash relations

## There are 0 SNP dropped because of low call rate!

## There are 0 sample set to NA because too little SNPs called!

## There are 0 SNP dropped because of low call rate!

## There are 0 sample set to NA because too little SNPs called!

## Using 227 polymophic SNPs to determine allele sharing.

## Running `rectangular` IBS algorithm!

## 133 of 333032 (0.04%) ...

## 13300 of 333032 (3.99%) ...

## 26600 of 333032 (7.99%) ...

## 39900 of 333032 (11.98%) ...

## 53200 of 333032 (15.97%) ...

## 66500 of 333032 (19.97%) ...

## 79800 of 333032 (23.96%) ...
```

```
## 93100 of 333032 (27.96%) ...

## 106400 of 333032 (31.95%) ...

## 119700 of 333032 (35.94%) ...

## 133000 of 333032 (39.94%) ...

## 146300 of 333032 (43.93%) ...

## 159600 of 333032 (47.92%) ...

## 172900 of 333032 (51.92%) ...

## 186200 of 333032 (55.91%) ...

## 199500 of 333032 (59.9%) ...

## 212800 of 333032 (63.9%) ...

## 226100 of 333032 (67.89%) ...

## 239400 of 333032 (71.88%) ...

## 252700 of 333032 (75.88%) ...

## 266000 of 333032 (79.87%) ...

## 279300 of 333032 (83.87%) ...

## 292600 of 333032 (87.86%) ...

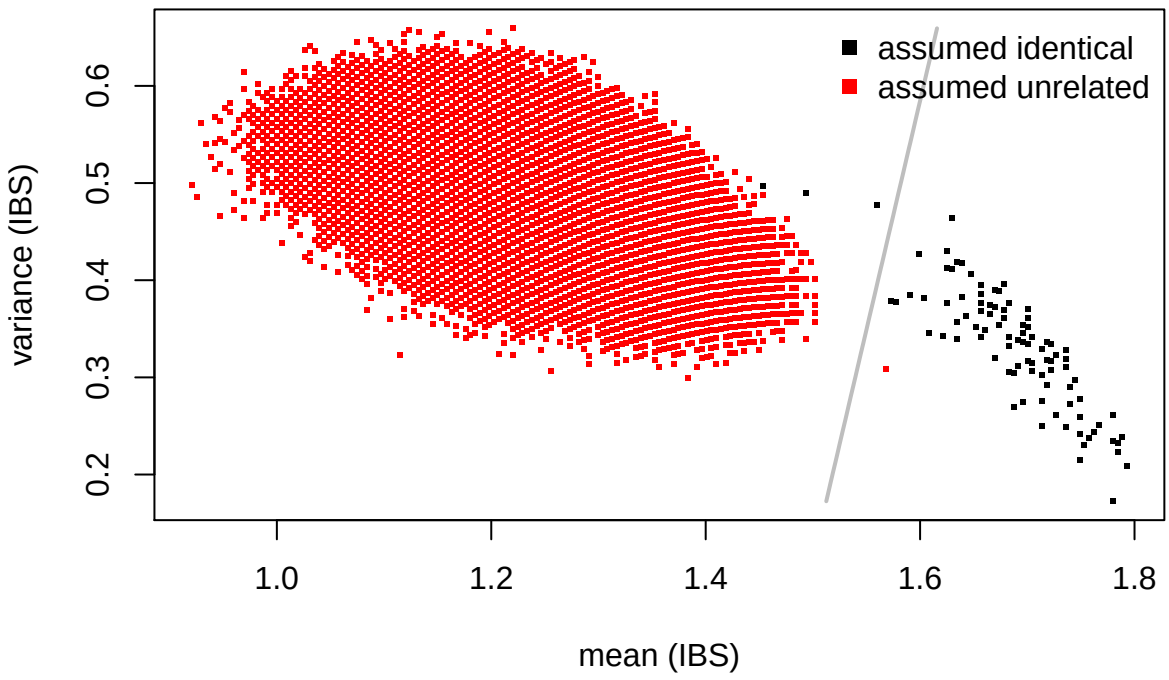
## 305900 of 333032 (91.85%) ...

## 319200 of 333032 (95.85%) ...

## 332500 of 333032 (99.84%) ...
```

```
mismatches <- inferRelations(data)
```

```
##                   Assumed relation
## Predicted relation identical unrelated
##          identical       100         1
##          unrelated         3    332928
```




```
mismatches
```

```
##            mean       var colnames.x colnames.y  relation predicted
## 222117 1.493392 0.4900004    NA11829    NA11829 identical unrelated
## 229339 1.559471 0.4776422    NA12716    NA12716 identical unrelated
## 229503 1.453744 0.4967448    NA12717    NA12717 identical unrelated
## 271889 1.568282 0.3083700    NA19128    NA19129 unrelated identical
```

```
require(readxl)
file <- tempfile(fileext=".ped")
download.file("ftp://ftp.1000genomes.ebi.ac.uk/vol1/ftp/technical/working/20130606_sample_info/20130606_

ped <- read.table(file, header=TRUE, sep="\t")
subset(ped, Individual.ID == "NA19129")
```

```
##      Family.ID Individual.ID Paternal.ID Maternal.ID Gender Phenotype
## 3468      Y077       NA19129     NA19128     NA19127      2         0
##      Population Relationship Siblings Second.Order Third.Order
## 3468        YRI        child        0            0           0
##      Other.Comments
## 3468              0
```

Two pairs of genotype-DNAm seems not properly matching. I suspect bad quality of DNAm or genotypes! And one unrelated is parent offspring relation, for example, this is verified by information from the pedigree file.

Detect mislabelled samples

In order to show how mislabelled samples can be detected some mislabeling is artificially introduced, i.e. a known `identical` relation is removed and `unrelated` pair is set to identical.

```
set.seed(12345)
relations <- expand.grid(idx = colnames(dnamCalls), idy = colnames(dnaCalls))
relations$relation_type <- "unrelated"
relations$relation_type[as.character(relations$idx) == as.character(relations$idy)] <- "identical"

id <- sample(which(relations$relation_type == "identical"), 1)
relations[id,]
```

```
##            idx     idy relation_type
## 271697 NA19119 NA19119     identical
```

```
relations$relation_type[id] <- "unrelated"

id <- sample(which(relations$relation_type == "unrelated"), 1)
relations[id,]
```

```
##            idx     idy relation_type
## 291674 NA12057 NA19707     unrelated
```

```
relations$relation_type[id] <- "identical"
relations <- relations[relations$relation_type == "identical",]

library(omicsPrint)
data <- alleleSharing(dnamCalls, dnaCalls, relations=relations, phasing=TRUE)
```

```
## Hash relations

## There are 0 SNP dropped because of low call rate!

## There are 0 sample set to NA because too little SNPs called!

## There are 0 SNP dropped because of low call rate!

## There are 0 sample set to NA because too little SNPs called!

## Using 227 polymophic SNPs to determine allele sharing.

## Running `rectangular` IBS algorithm!

## 133 of 333032 (0.04%) ...

## 13300 of 333032 (3.99%) ...

## 26600 of 333032 (7.99%) ...
```

```
## 39900 of 333032 (11.98%) ...

## 53200 of 333032 (15.97%) ...

## 66500 of 333032 (19.97%) ...

## 79800 of 333032 (23.96%) ...

## 93100 of 333032 (27.96%) ...

## 106400 of 333032 (31.95%) ...

## 119700 of 333032 (35.94%) ...

## 133000 of 333032 (39.94%) ...

## 146300 of 333032 (43.93%) ...

## 159600 of 333032 (47.92%) ...

## 172900 of 333032 (51.92%) ...

## 186200 of 333032 (55.91%) ...

## 199500 of 333032 (59.9%) ...

## 212800 of 333032 (63.9%) ...

## 226100 of 333032 (67.89%) ...

## 239400 of 333032 (71.88%) ...

## 252700 of 333032 (75.88%) ...

## 266000 of 333032 (79.87%) ...

## 279300 of 333032 (83.87%) ...

## 292600 of 333032 (87.86%) ...

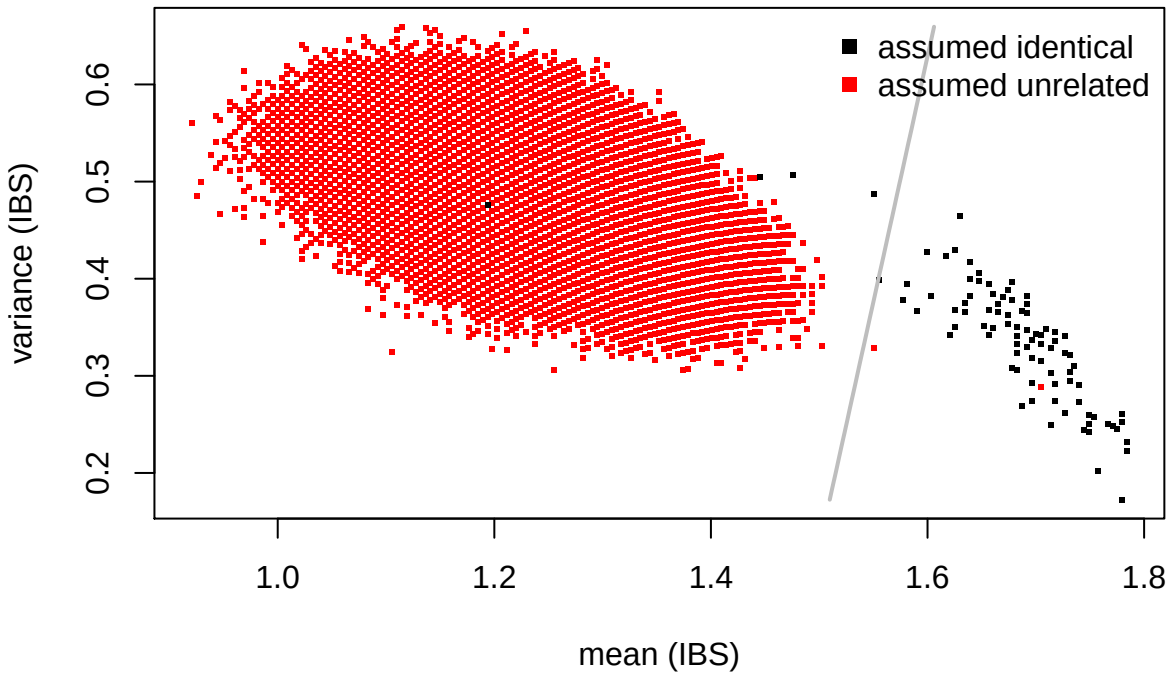
## 305900 of 333032 (91.85%) ...

## 319200 of 333032 (95.85%) ...

## 332500 of 333032 (99.84%) ...
```

```
mismatches <- inferRelations(data)
```

```
##                    Assumed relation
## Predicted relation identical unrelated
##          identical        99         2
##          unrelated         4    332927
```




```
mismatches
```

```
##            mean       var colnames.x colnames.y  relation predicted
## 222117 1.475771 0.5071537    NA11829    NA11829 identical unrelated
## 229339 1.550661 0.4874664    NA12716    NA12716 identical unrelated
## 229503 1.444934 0.5046977    NA12717    NA12717 identical unrelated
## 271697 1.704846 0.2886047    NA19119    NA19119 unrelated identical
## 271889 1.550661 0.3281743    NA19128    NA19129 unrelated identical
## 291674 1.193833 0.4755370    NA12057    NA19707 identical unrelated
```

These mislabel samples (pairs) are automatically detected.

SessionInfo

```
sessionInfo()
```

```
## R version 3.3.2 (2016-10-31)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 16.04.2 LTS
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=en_US.UTF-8        LC_COLLATE=en_US.UTF-8
##  [5] LC_MONETARY=en_US.UTF-8    LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=en_US.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
##  [1] readxl_1.0.0
##  [2] BiocParallel_1.8.2
##  [3] DNAmArray_0.0.2
##  [4] minfi_1.20.2
##  [5] bumphunter_1.14.0
##  [6] locfit_1.5-9.1
##  [7] iterators_1.0.8
##  [8] foreach_1.4.3
##  [9] Biostrings_2.42.1
## [10] XVector_0.14.1
## [11] SummarizedExperiment_1.4.0
## [12] FDb.InfiniumMethylation.hg19_2.2.0
## [13] org.Hs.eg.db_3.4.0
## [14] TxDb.Hsapiens.UCSC.hg19.knownGene_3.2.2
## [15] GenomicFeatures_1.26.4
## [16] AnnotationDbi_1.36.2
## [17] GenomicRanges_1.26.4
## [18] GenomeInfoDb_1.10.3
## [19] IRanges_2.8.2
## [20] S4Vectors_0.12.2
## [21] GEOquery_2.40.0
## [22] Biobase_2.34.0
## [23] BiocGenerics_0.20.0
## [24] omicsPrint_0.99.8
## [25] MASS_7.3-47
## [26] BiocStyle_2.2.1
##
## loaded via a namespace (and not attached):
##  [1] httr_1.2.1               nor1mix_1.2-2
##  [3] bit64_0.9-7              splines_3.3.2
##  [5] doRNG_1.6.6              blob_1.1.0
##  [7] cellranger_1.1.0         Rsamtools_1.26.2
##  [9] yaml_2.1.14              RSQLite_2.0
## [11] backports_1.1.0          lattice_0.20-35
```

```
## [13] quadprog_1.5-5           limma_3.30.13
## [15] digest_0.6.12            RColorBrewer_1.1-2
## [17] preprocessCore_1.36.0    htmltools_0.3.6
## [19] Matrix_1.2-10            plyr_1.8.4
## [21] siggenes_1.48.0          XML_3.98-1.9
## [23] pkgconfig_2.0.1          biomaRt_2.30.0
## [25] genefilter_1.56.0        zlibbioc_1.20.0
## [27] xtable_1.8-2             RevoUtilsMath_10.0.0
## [29] annotate_1.52.1          openssl_0.9.6
## [31] tibble_1.3.3             beanplot_1.2
## [33] pkgmaker_0.22            survival_2.41-3
## [35] magrittr_1.5             mclust_5.3
## [37] memoise_1.1.0            evaluate_0.10.1
## [39] nlme_3.1-131             data.table_1.10.4
## [41] tools_3.3.2              registry_0.3
## [43] matrixStats_0.52.2       stringr_1.2.0
## [45] rngtools_1.2.4           base64_2.0
## [47] rlang_0.1.1              grid_3.3.2
## [49] RCurl_1.95-4.8           bitops_1.0-6
## [51] rmarkdown_1.6            codetools_0.2-15
## [53] multtest_2.30.0          DBI_0.7
## [55] reshape_0.8.6            R6_2.2.2
## [57] illuminaio_0.16.0        GenomicAlignments_1.10.1
## [59] knitr_1.16               rtracklayer_1.34.2
## [61] bit_1.1-12               rprojroot_1.2
## [63] stringi_1.1.5            Rcpp_0.12.11
```

Reference

Chen, Y. A., M. Lemire, S. Choufani, D. T. Butcher, D. Grafodatskaya, B. W. Zanke, S. Gallinger, T. J. Hudson, and R. Weksberg. 2013. "Discovery of cross-reactive probes and polymorphic CpGs in the Illumina Infinium HumanMethylation450 microarray." *Epigenetics* 8 (2): 203–9.

Moen, E. L., X. Zhang, W. Mu, S. M. Delaney, C. Wing, J. McQuade, J. Myers, L. A. Godley, M. E. Dolan, and W. Zhang. 2013. "Genome-wide variation of cytosine modifications between European and African populations and the implications for complex traits." *Genetics* 194 (4): 987–96.

Zhou, W., P. W. Laird, and H. Shen. 2016. "Comprehensive characterization, annotation and innovative use of Infinium DNA methylation BeadChip probes." *Nucleic Acids Res.*, Oct.